Project Report

<u>Topic:</u>

TIAGO ROBOT ROS2

(Pick and Place Object with Navigation and Mapping)

Submitted By:

Nirbhay Borikar
(Master Mechatronics)
Specializing: Intelligent Robotics

Ravensburg Weingarten University of Applied Science ,Germany

Table of Contents:

Serial No.	Topic	Page No.
1	Introduction	3
2	Algorithm Development for Pick and Place 2.1 Developer Environment 2.2 SLAM 2.3 Using Ros2 Topic for Navigation 2.4 AprilTag and Transformation 2.5 Robot Manipulation Framework 2.7 Adding Collision Objects 2.8 Pick and Place	4-12
3	Package Development Aspects: CMakelists , Multithreading and Python	13
4	Evaluation	14
5	Conclusion	15

Chapter 1

Introduction

This project focuses on the manipulation, navigation, and planning of the TIAGO robot using ROS2 and robot interaction framework PyMoveIt2. The system is broken down into multiple tasks that progressively build upon each other to enable the autonomous movement and manipulation capabilities of the robot.

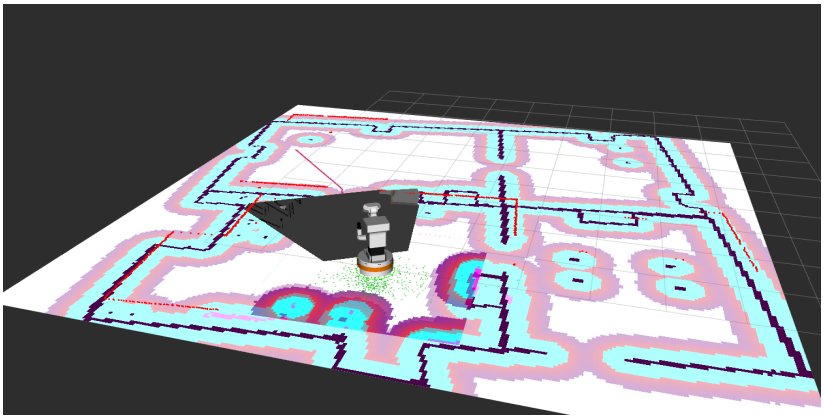

Fig 1: Mapping and Navigation.

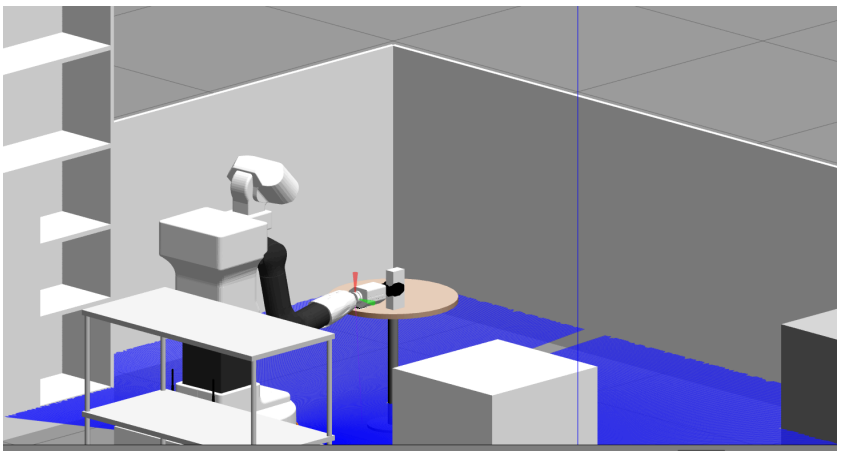

Fig 2: Manipulation of Tiago Robot.

The key objectives include:

- ROS2 Navigation: Enabling the TIAGO robot to autonomously move within a simulated environment using SLAM (Simultaneous Localization and Mapping) and the Navigation Stack.
- AprilTag Tracking: Implementing AprilTag detection and TF transformations to determine the robot's position relative to objects and waypoints.
- Collision Object Handling: Adding virtual obstacles in the simulation environment to ensure safe motion planning using PyMoveIt2.
- Pick and Place Operation: Using PyMoveIt2 for grasping and placing objects based on real-time pose estimation. (All task action videos—> here)

Chapter 2

Algorithm Development for Pick and Place

The following algorithm was implemented in this task to achieve the desired results.

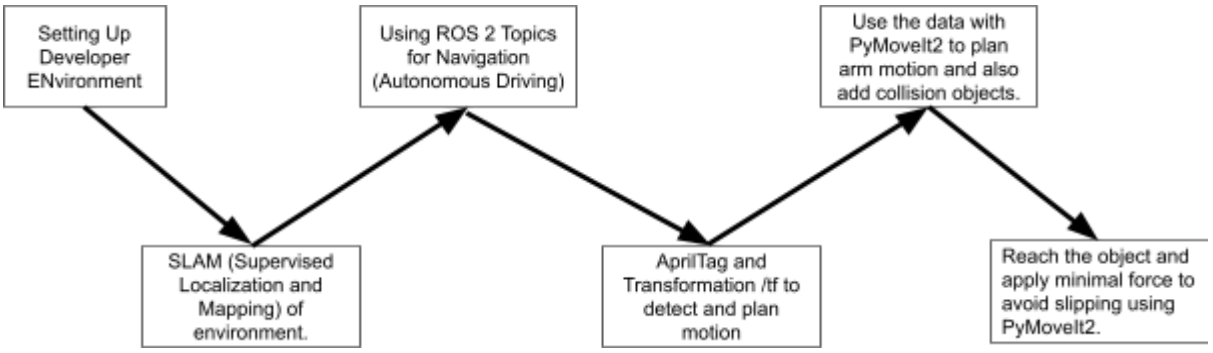

Fig 3: Steps followed in the algorithm

2.1 Developer Environment: (Linux, Docker, ROS 2, Gazebo, Rviz)

Linux operating system due to its strong compatibility with robotics frameworks and open-source tools. The development environment for the TIAGo robot pick-and-place project was built on it.

Docker is a container management technology of packages containing codes, dependencies, libraries and more. On top of Linux, it was used to containerize the ROS 2 workspace, ensuring reproducibility, easier dependency management, and isolation of simulation, visualization, and development components. This setup helped streamline the development process and avoid conflicts across systems.

ROS 2 (Robot Operating System 2) : A robotics middleware with libraries and tools for robot applications. This served as the core framework for building TIAGO robot control, perception, and navigation logic. ROS 2 packages were developed and deployed inside Docker containers using a “docker compose.yml” configuration, which allowed separate nodes like the Gazebo simulation, robot control, and RViz interface to run concurrently in an isolated, yet connected environment.

Gazebo, a robot simulation tool integrated with ROS. In this 3D environment the robot's movement and sensor data collection is evaluated. Created a simulated environment hallways, shelves, table and other objects in different rooms in the environment.

Rviz, short form of ROS Visualization, is used to see the robot sensor data, map, also for visualizing coordinate frames of robots and path that robot is following.

2.2 SLAM (Supervised Localization and Mapping)

Simultaneous Localization and Mapping, with this technique used for building a map of an unknown environment by the robot while localizing itself in the map, without using additional peripherals with the robot such as GPS.

SLAM components deployed using docker containers, and by manually driving the robot with rqt_robot_steering which publishes standard messages (Twist) of movements on /cmd_vel topic of ROS 2. The map was successfully saved and linked in docker files or compose template , to drive the robot on the specific map environment.

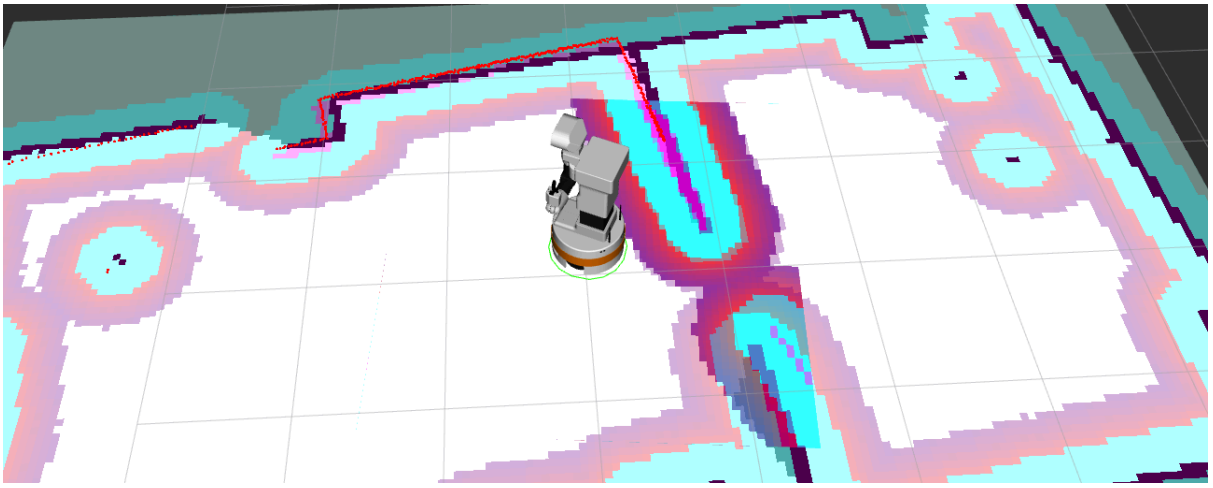

Fig 4: Mapping of the environment

General Commands Used:

1. ros2 run rqt_robot_steering rqt_robot_steering (Open Steering)
2. ros2 run nav2_map_server map_saver_cli -f myfirstmap (For map saving)

2.3 Using Ros2 Topic for Navigation

Activated navigation components and drive robots on previously generated maps. The topic /initialpose to set the robot starting position using the message type (geometry_msgs/msg/PoseWithCovarianceStamped). So whenever the Simulation starts and the node containing initialization of the /intialpose topic will immediately place the robot to the required position, before starting the navigation. Furthermore, the topic /goal_pose is used with message type “PoseStamped”, moving robot from one position to another it’s a long running task hence “Action Server” (nav2_msgs/action/NavigateToPose) used as it provide real time feedback, it is different from “Action Service”, which is used for short, and for the on demand execution of a task, no feedback, just result of completion or rejection received.

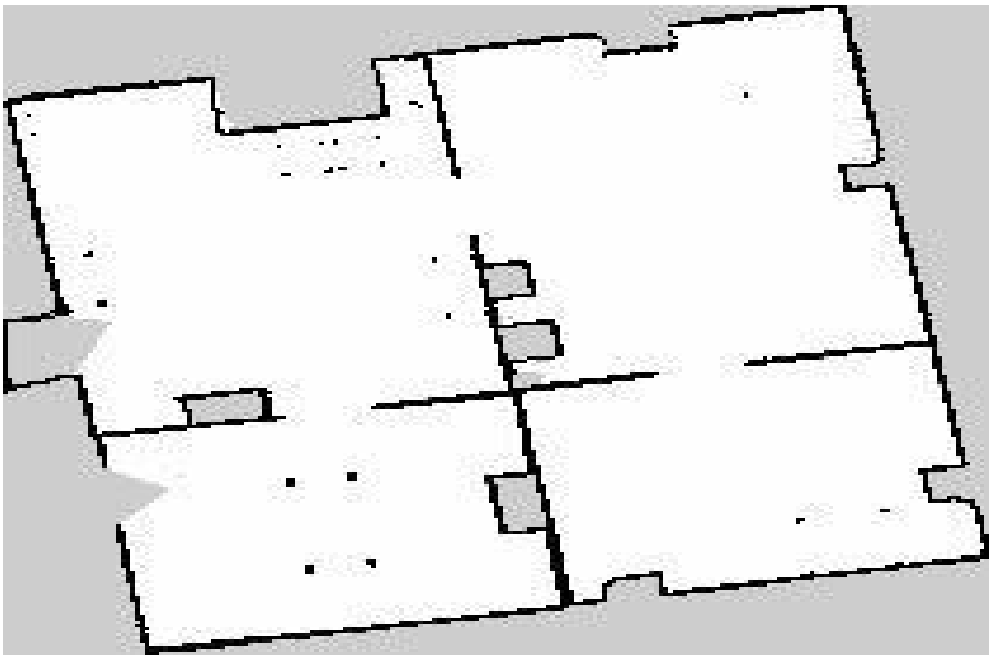

Fig 5: Map of the TIAGO Robot environment.

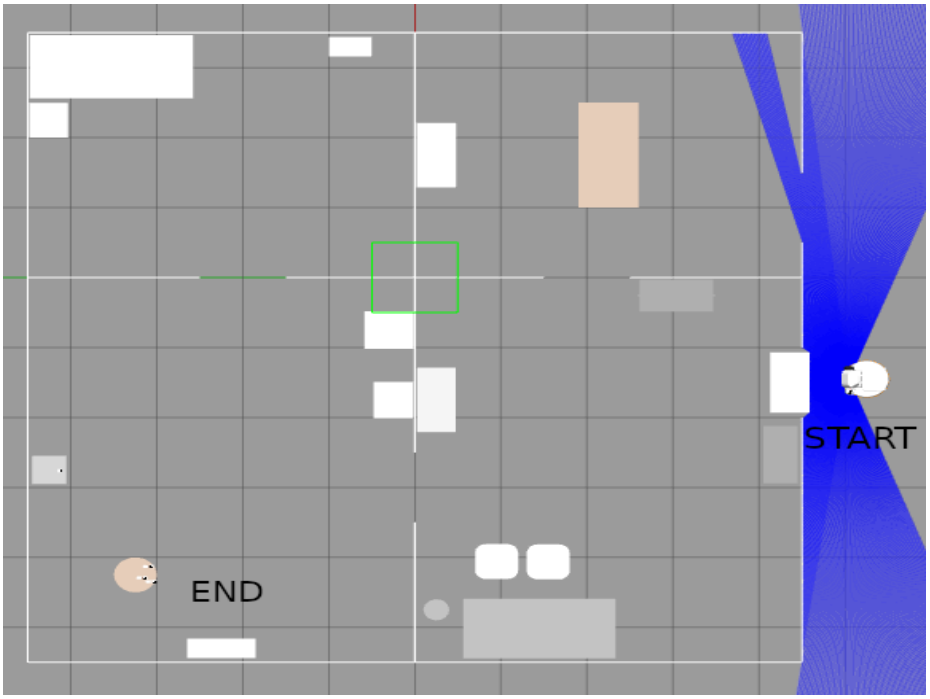

Fig 6: The image shows inspection to desired goal by using ROS2 topic

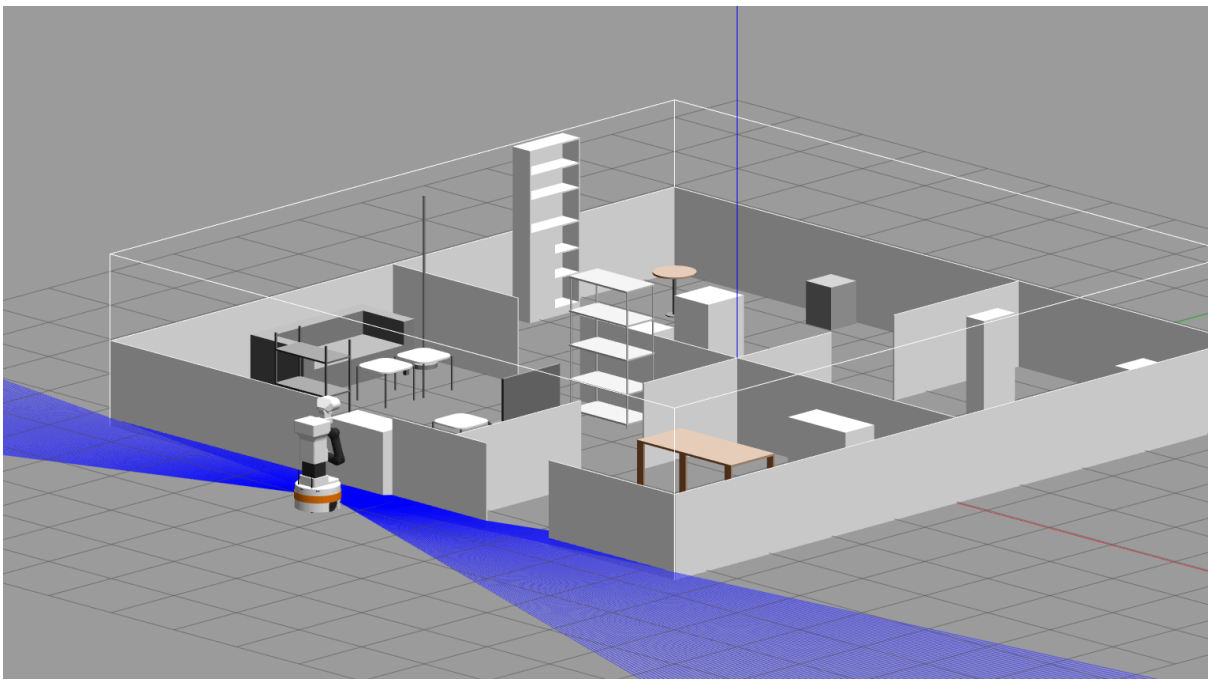

Fig 7: Robot in initial position.

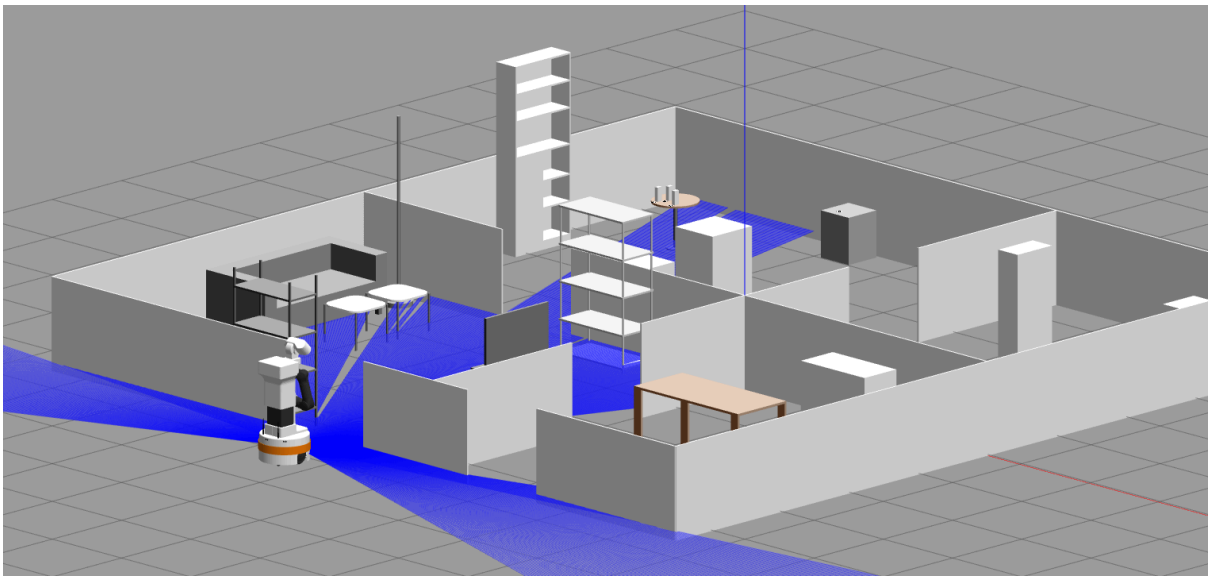

Fig 8: Door Opened manually the robot should detect the door opened and should execute its plan.(Click here for video)

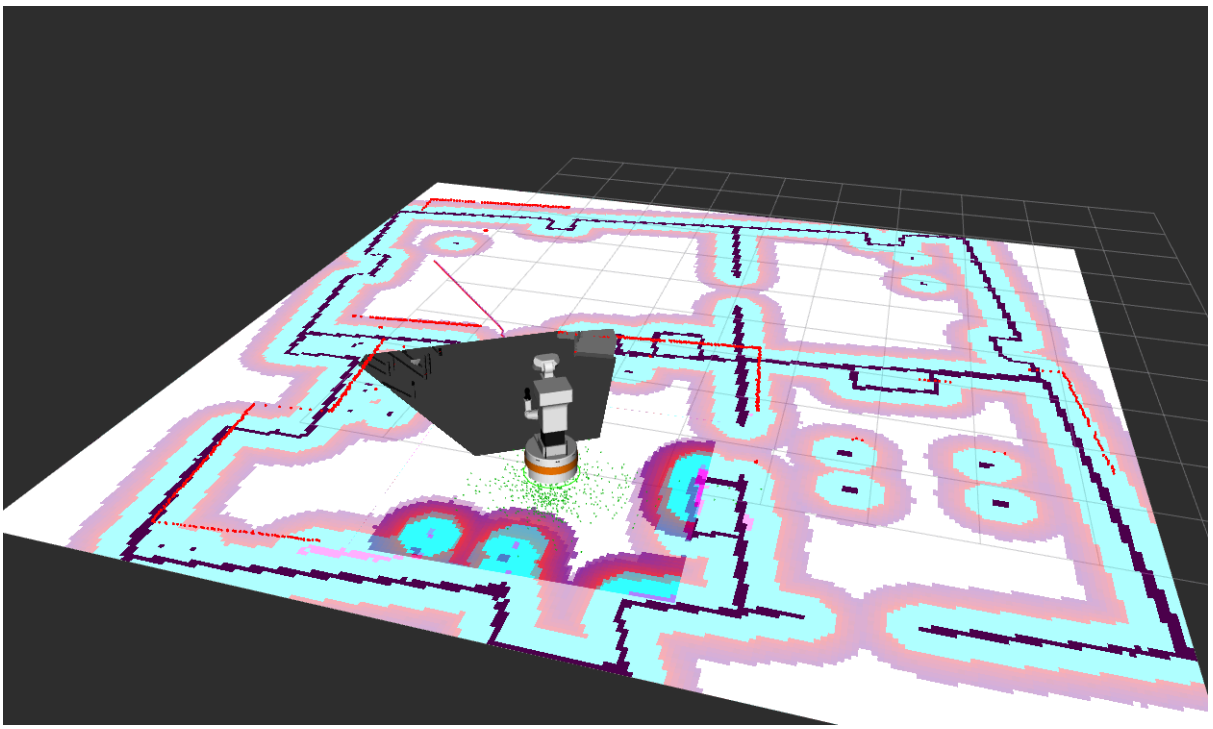

Fig 9: Navigation to desired destiny started.

2.4 AprilTag and Transformation

Utilized AprilTag, in order to detect objects that need to be grasped , and with /tf (transformation) pose and orientation of the coordinate or frame is stored or published in the ROS2 topics

AprilTag, visual fiducial marker that is used for detecting objects in Robotics systems. It is used here in detecting objects that need to be grabbed, a table where the object was placed.

Below is the visual representation of AprilTag in the Rviz environment which is getting detected by a camera sensor in TIAGO Robot.

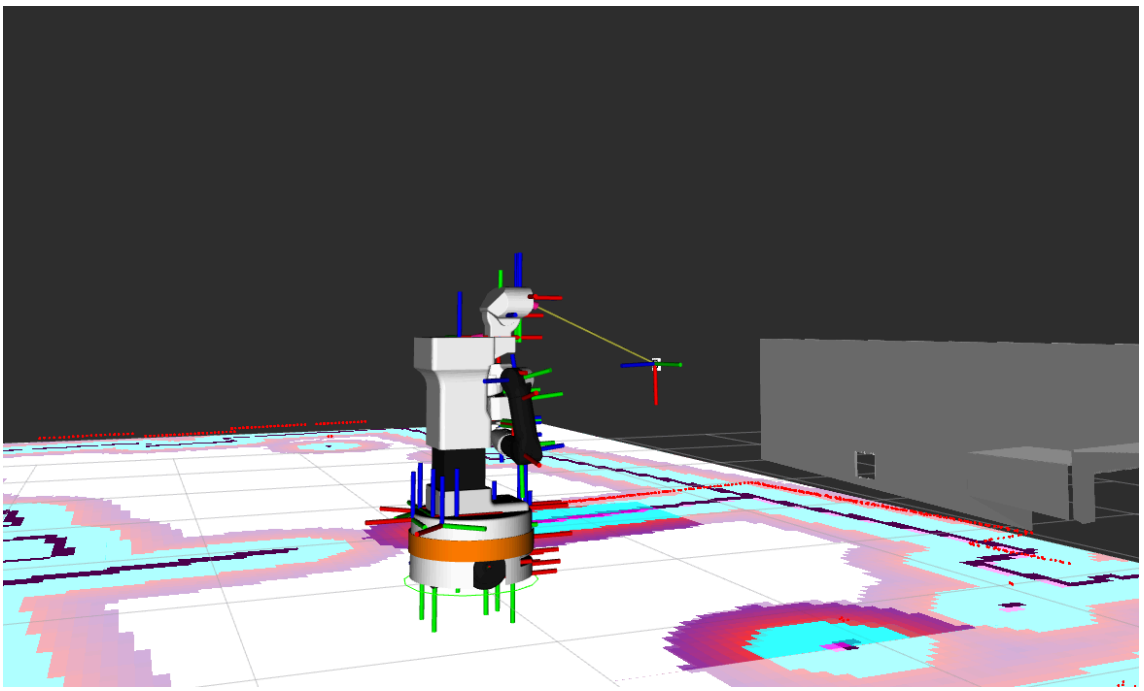

Fig 10: Visualizing the AprilTag.

TF (Transform Library) it's a library used in ROS for representing and managing the relationships between different coordinate frames in a robotic system. Allows to express the position and orientation of one frame relative to another using a transformation matrix. The matrix contains- Translation (x, y, z): The distance between the origins of the two frames and Rotation (roll, pitch, yaw).

Here the information of coordinates is subscribed in the /detections topic of ROS. Tf2 library upgrade was used, it has "tf2_listener" and "Broadcaster", first listens to /tf and cache all data that it collected up to its maximum catch limit. Second, it publishes the transform between coordinates on topics.

Below image shows a visual representation after detecting AprilTag pose, how the arm is reached to exact position. (Movement of arm deployed using PyMoveIt2 a robot interaction framework).

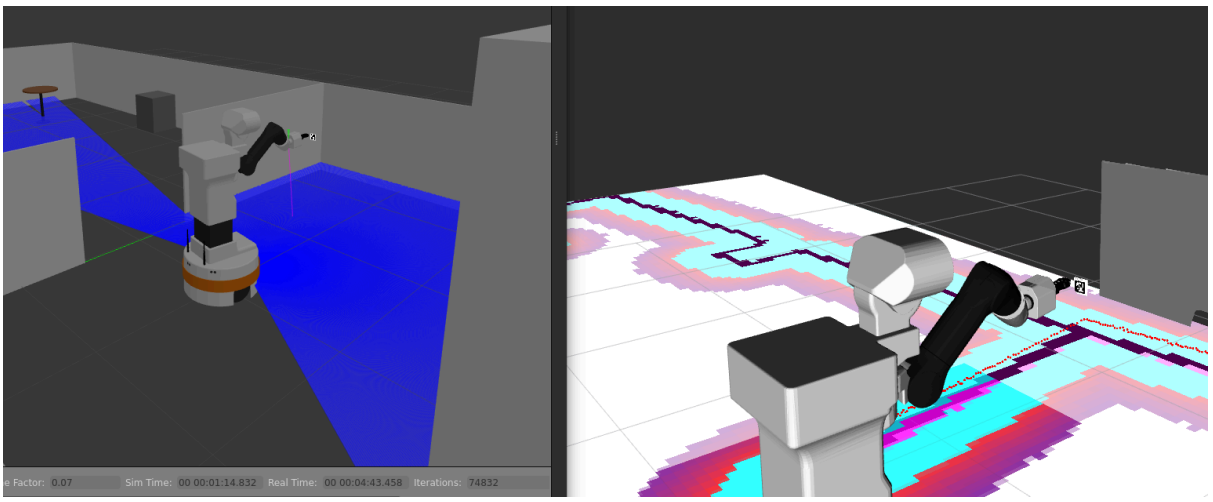

Fig 11: Tracking AprilTag, collecting /tf and moving arm using Pymoveit2. ([Click here for video])

2.5 Robot Manipulation Framework : PyMovieIt2

To implement a ROS2-based system for the Tiago robot to move its hand using PyMoveIt2 framework to plan its arm movement to desired location by collecting desired location data by using Apriltag and extracting pose from transformation (tf) and picking the object in the table to place somewhere else.

PyMoveIt2: A robotics manipulation framework that helps in moving a robot, here it used to operate the Tiago Arm in order to move to desired position and grab the object and place it.

PyMoveIt2 , have high level control for move to pose, joint targets, or to follow a cartesian path, while in C++ MoveIt2 , this is more complex to integrate in few lines. Furthermore, PyMoveIt2 figure out how to move safely the arm without crashing into things using collision objects involvement in the environment, TIAGO Robot has right URDFs and config files to work well with MoveIt2, hence this framework is the suitable choice.

2.6 Adding Collision Object

In Gazebo and Rviz using pymoveit2, a **Collision Object** is used to define obstacles or environmental constraints in a robotic workspace. This is crucial for motion planning, ensuring the robot avoids collisions while executing movements. A Collision Object is typically represented as a geometric shape (e.g., box, sphere, cylinder, or mesh) with a defined position and orientation in the world. These objects help in path planning by informing the motion planner about restricted regions in the environment. It requires a TF (Transform) to be properly placed in the simulation environment. In this project case collision object is added so that the robot will not hit that object while performing movement

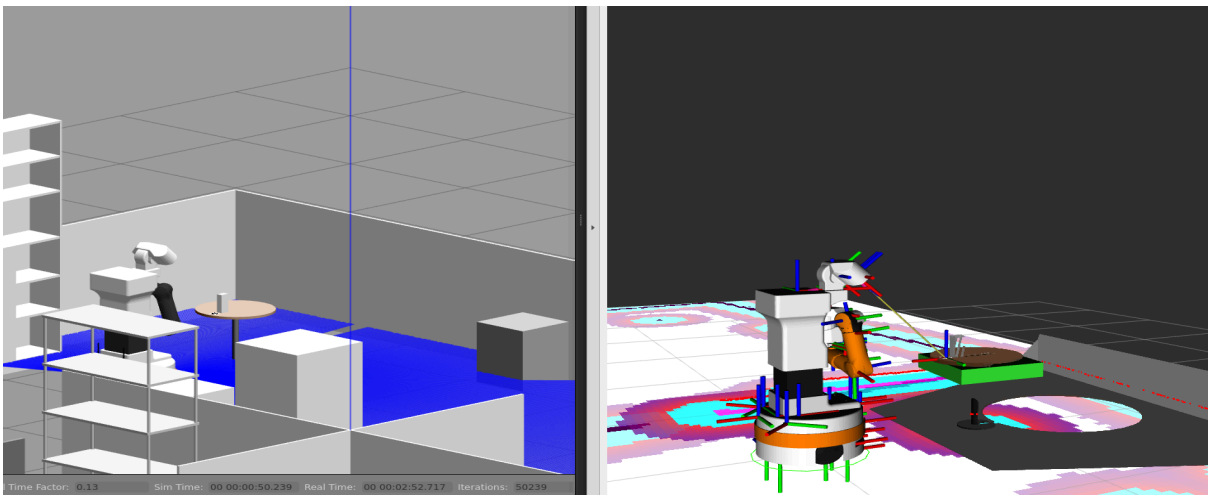

Fig 12: Collision object added below the table using Pymoveit2.

2.7 Pick and Place

Manipulation: After processing pose data from /tf the arm movement was tested using the PyMoveIt2 standard functions. One special approach deployed here is that the transformation data was stored in a separate variable, because when vision range is blocked the real time transformation data will not be published by the broadcaster, possibly because when the robot hand comes in front of camera vision and blocks its vision.

Below you can see the arm is blocking vision ,and the arm is still moving with the help of stored information.

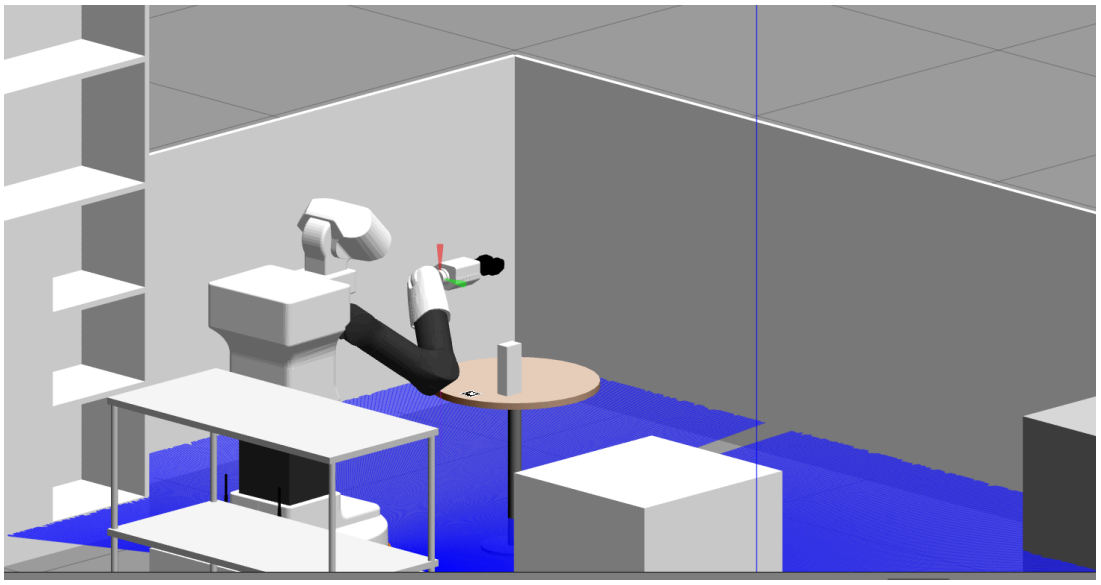

Fig 18: The robot arm motion planning executing. (The robot arm motion is executed by PyMoveIt 2 function.)

Below the image shown describe the precision of TIAGO Robot to grab the object, its closing gripper with the help of standard PyMoveIt2 function “close gripper”. It applies minimal force, so that the object is not slight from the gripper.

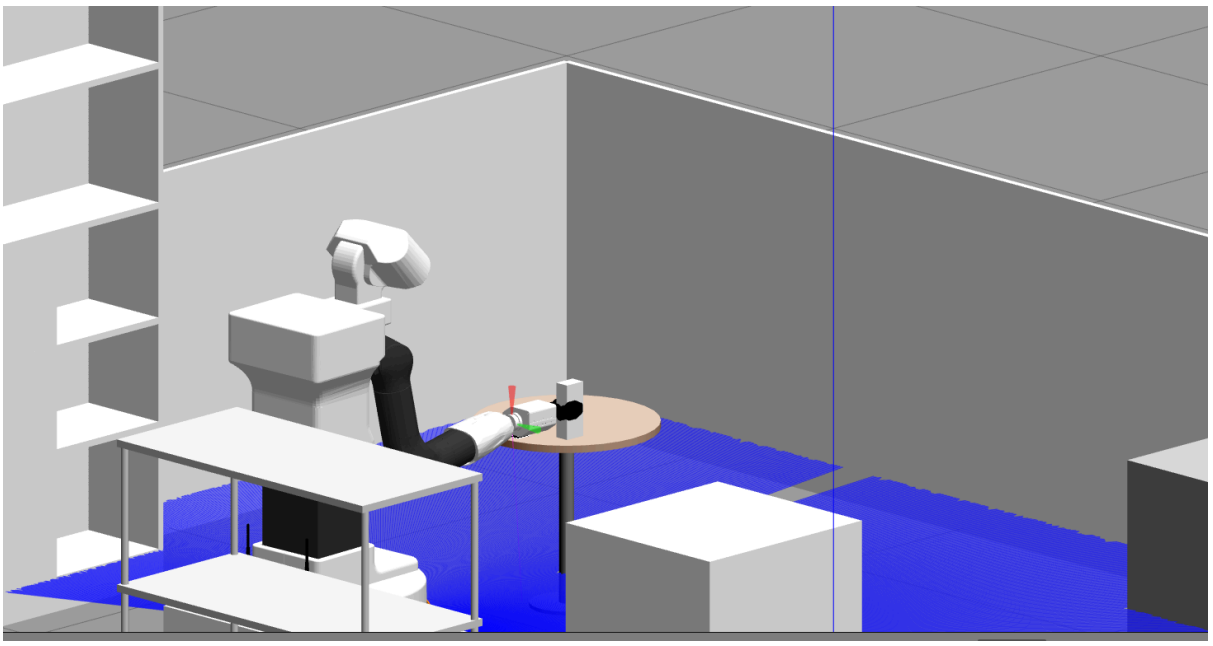

Fig 19: The robot arm grabbed the object. (Click here for video)

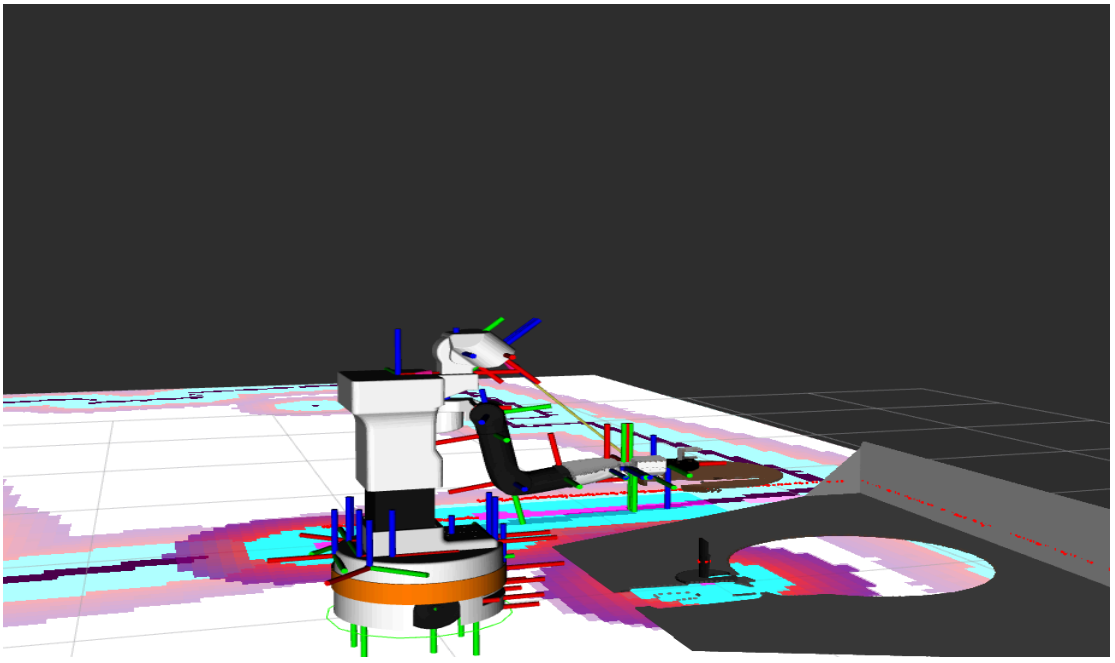

Fig 20: Tf view while holding the object and removing the collision object .

Chapter 3

Package Development Aspects: CMakelists, Multithreading, Python**.**

In the development of ROS2 packages for the TIAGO robot pick-and-place, proper structuring and optimization of the codebase were essential for achieving real time performance, scalability, and modular integration. Three core aspects that played a crucial role in this context were the use of CMakeLists.txt, multithreading, and Python scripting.

The **CMakeLists.txt** file is a fundamental component in any ROS2 package, used to configure the build system. It defines how the code is compiled, which dependencies to include, and how nodes are linked. In this project, CMakeLists.txt was used to integrate both Python and C++ nodes, ensuring proper inclusion of libraries such as rclcpp, ament_cmake, MoveIt2, and sensor_msgs. It also allowed registration of launch files, Python scripts, and dynamic dependencies like Gazebo plugins and URDF files(Robot physical description XML file.)

Multithreading was crucial for handling concurrent processes, such as listening to transformation frames (tf2_listener), executing motion plans, and monitoring feedback from navigation or action servers. In ROS2, the MultiThreadedExecutor was used to allow multiple callbacks such as AprilTag detection, motion planning, and gripper control to be processed in parallel. In this project while one thread handled continuous tf broadcasting for the detected AprilTag, another simultaneously executed a planned trajectory, making the system more responsive and reducing latency in pick-and-place execution.

Python was chosen extensively for high-level control due to its simplicity and rapid development cycle, especially with frameworks like PyMoveIt2. Unlike C++, which has more constraints for ROS2 integration, Python allows direct and intuitive manipulation of robot arms using just a few lines of code. In this project, Python scripts were used for: Sending commands to the robot arm (pose), Processing TF transformations from AprilTag detections, Defining collision objects dynamically during execution and Handling action clients for navigation and gripper control. Additionally, Python was useful for logging, debugging, and integrating ROS2 launch files, which facilitated modular and reusable experiment setups.

Chapter 4

Evaluation

Collision Object: The collision object position changes because the apriltag fickle they are just floating in the environment not attached to the object tightly, and hence difference in transformation pose.

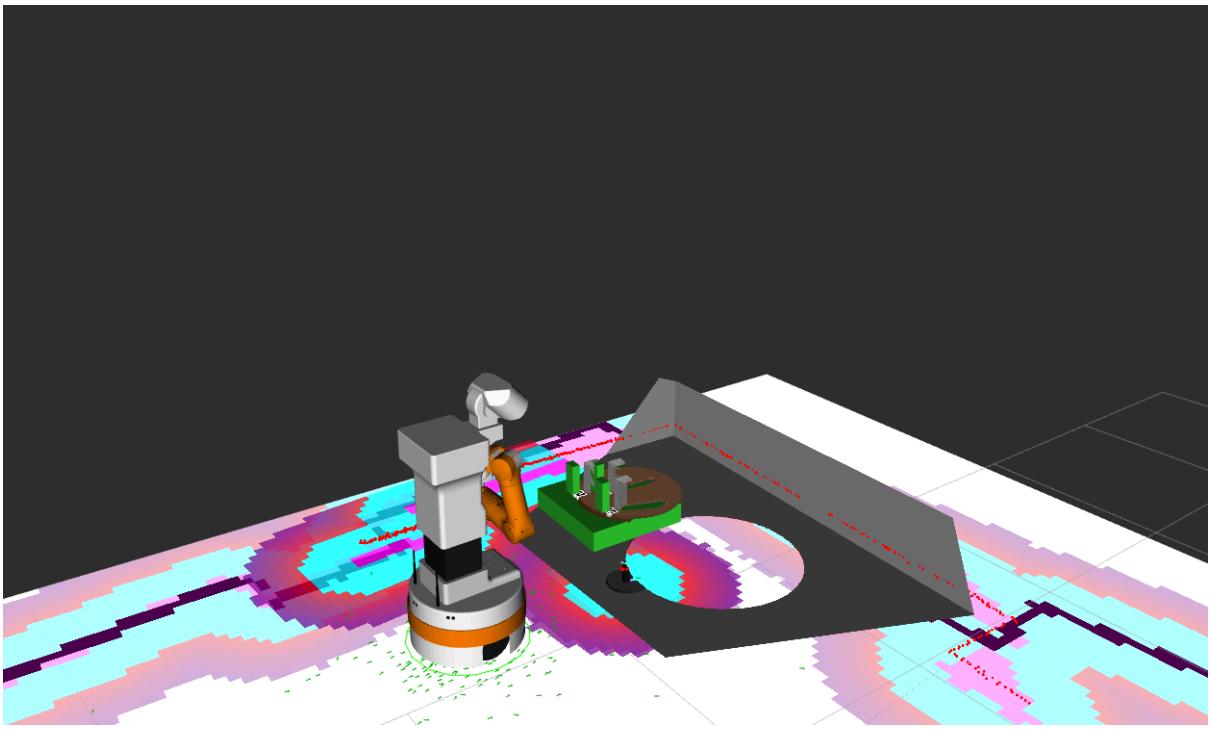

Fig : Collision object in 3 rectangular object, and also in table added.(With Deformation)

Move Arm to Pose:

Object Slipping from the hand during grasping: The gripper closes but does not apply force, causing the object to slip. Friction and contact stability between the gripper and object are not sufficient. PyMoveIt 2 Sometimes Fails Motion Planning (Invalid Trajectory): MoveIt2 randomly samples the start state for motion planning. If the start position is too close to joint limits (angle or position they can move) or singularities, the planner fails.

Robot Hand Sometimes Hit the Object in PyMoveIt2 : This happens due to incorrect motion planning, collision detection issues, or bad grasp approach angles. The arm moves directly to the object instead of approaching from a safe height. This causes unexpected collisions with the object.

Chapter 5

Conclusion

This project successfully demonstrated the use of SLAM, Visualization and Manipulation in a ROS2-based system. By integrating AprilTag detection, TF transformations, and PyMoveIt2 based motion planning, the Tiago robot was able to detect, navigate, grasp, and place objects with a great degree of precision.

However, despite the successful implementation, precision and accuracy issues were encountered due to the inherent challenges of motion planning and grasping mechanics in PyMoveIt2. Future work, in order to encounter collision object issues, we can move the robot a little closer to the table so that this virtual object pops up at approximately the same position of the physical object, which leads to better results.

Manipulation issues, for the slipping problem, ensure that the gripper's force parameters are correctly configured, and that grasp planning includes sufficient friction and contact surface with the object. In some cases, adjusting the gripper's control mode (from position to effort or torque control) may help it apply and maintain the necessary holding force. To address motion planning failures due to joint limits or singularities, you can provide a well-defined, valid start state to the planner rather than relying on MoveIt2's random sampling. This improves planning consistency and reduces the chance of invalid trajectories. Additionally, using joint constraint filters or smoothing tools can help avoid problematic configurations. Finally, to prevent collisions between the robot hand and the object, it's crucial to define a clear pre-grasp approach strategy—typically involving a top-down approach from a safe height, followed by a vertical descent. Ensure that collision objects are correctly modeled in the planning scene and that the motion planner accounts for them during trajectory generation. Adding waypoints or Cartesian paths can further refine the approach to ensure safe and predictable motion.

At last, the overall system successfully demonstrated the integration of vision-based object detection with motion planning, paving the way for advanced robotic manipulation tasks in dynamic environments. (Click here for video)